

1) Mecanismos de Protección de acceso al Segmento en Modo Protegido

a. Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente como interpreta un valor en un registro de segmento si TI=1 o TI=0.

El 80386 utiliza un selector de segmento y una tabla de descriptores para realizar la traducción de direcciones lógicas a direcciones lineales, implementando el mecanismo de protección. El no acceso directo al segmento se refiere a que la dirección base del segmento no se almacena directamente en el registro de segmento, sino que se obtiene de una tabla, lo que permite la verificación.

El proceso de traducción es el siguiente:

1. El registro de segmento (como CS, DS, SS, ES, FS, GS) almacena el Selector de Segmento.
2. El Selector se descompone en tres partes:
 - Índice (Index): Indica la entrada (el descriptor) dentro de la tabla.
 - Indicador de Tabla (TI - Table Indicator): Determina qué tabla usar.
 - Nivel de Privilegio Requerido (RPL - Requested Privilege Level): Indica el nivel de privilegio del código que intenta acceder al segmento.
3. La dirección del descriptor se calcula sumando el Registro Base de la Tabla (GDTR o LDTR) con el Índice multiplicado por el tamaño de la entrada (8 bytes).
4. Se obtiene el Descriptor de Segmento de la tabla (GDT o LDT).
5. El Descriptor contiene la Dirección Base (Base Address) real del segmento y los Derechos de Acceso (Access Rights).
6. Finalmente, la Dirección Lineal se forma sumando la Dirección Base con el Desplazamiento (Offset).

Interpretación del bit TI (Table Indicator)

El bit TI (Table Indicator) del Selector de Segmento determina qué tabla de descriptores se debe utilizar:

- TI = 0 (Tabla Global): Indica que el descriptor del segmento se encuentra en la Tabla de Descriptores Globales (GDT - Global Descriptor Table). La GDT contiene descriptores para segmentos accesibles a todos los procesos del sistema (ej. código del kernel, estructuras del SO).
- TI = 1 (Tabla Local): Indica que el descriptor del segmento se encuentra en la Tabla de Descriptores Locales (LDT - Local Descriptor Table). La LDT contiene descriptores para segmentos específicos de un proceso o tarea (ej. su propio código, pila, datos).

b. ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?

Para que un selector pueda acceder a un segmento, se deben cumplir tres condiciones fundamentales de validación y protección:

1. Validación del Descriptor: El descriptor del segmento debe ser Válido. Esto se verifica principalmente con el bit P (Presente), que debe ser $\text{P}=1$. Si $\text{P}=0$, se genera una excepción de fallo de segmento (Segment Not Present).
2. Verificación del Límite (Limit Check): El desplazamiento (offset) dentro del segmento que se intenta acceder debe ser menor o igual al límite (limit) del segmento definido en el descriptor.
3. Verificación del Nivel de Privilegio (Privilege Check): Este es el corazón de la protección de memoria y garantiza que un código con bajo privilegio no pueda acceder a datos o código de alto privilegio.

Relación entre RPL y DPL

El mecanismo de control de acceso utiliza cuatro niveles de privilegio (Rings), de 0 (máximo privilegio, Kernel) a 3 (mínimo privilegio, Aplicaciones). Los registros clave en esta verificación son:

- RPL (Requested Privilege Level - Nivel de Privilegio Requerido): Son los dos bits menos significativos del Selector de Segmento. Indica el nivel de privilegio de la entidad que intenta hacer la solicitud de acceso (el código llamante).
- DPL (Descriptor Privilege Level - Nivel de Privilegio del Descriptor): Son los dos bits en el Descriptor de Segmento. Indica el nivel de privilegio mínimo que se necesita para acceder a ese segmento.

La regla general de protección para el acceso a datos (segmentos DS, ES, SS, etc.) es:

$$\text{Máximo}(\text{CPL}, \text{RPL}) \leq \text{DPL}$$

Donde CPL (Current Privilege Level) es el nivel de privilegio del código que se está ejecutando actualmente (almacenado en los bits 0 y 1 del registro de segmento CS).

- Acceso Permitido: Si el nivel de privilegio es igual o numéricamente menor (más privilegiado) que el DPL, el acceso es permitido.
- Acceso Denegado: Si el nivel de privilegio es numéricamente mayor (menos privilegiado) que el DPL, se genera una excepción de protección general (General Protection Fault).

En resumen: un proceso de bajo privilegio no puede acceder directamente a un segmento de datos de alto privilegio.

c. Explique que son y para qué sirven los siguientes registros y bits: GDTR, GDTR, RPL, LDTR, bit G, bit D, bit P, bit S, bit E, bit A.

Elemento	Tipo	Descripción	Propósito / Función
GDTR	Registro (48 bits)	Global Descriptor Table Register.	Almacena la Dirección Base (32 bits) y el Límite (16 bits) de la Tabla de Descriptores Globales (GDT) en memoria. Necesario para ubicar la GDT y realizar la traducción con TI=0.

Elemento	Tipo	Descripción	Propósito / Función
GDTR	Campo (16 bits)	Global Descriptor Table Limit.	Es la parte del registro GDTR que indica el tamaño máximo (límite) de la GDT. Se usa para verificar que el índice del selector no exceda la tabla.
LDTR	Registro (16 bits)	Local Descriptor Table Register.	Contiene el Selector para el descriptor de la Tabla de Descriptores Locales (LDT). Permite al SO cambiar la LDT para cada proceso.
RPL	Bits (2 bits)	Requested Privilege Level.	Bits 0 y 1 del Selector de Segmento. Indica el nivel de privilegio solicitado para la operación de acceso al segmento.
Bit G	Bit (1 bit)	Granularity bit (Granularidad).	En el Descriptor de Segmento. Si G=0, el límite del segmento se interpreta en unidades de byte (hasta 1 MB). Si G=1 el límite se interpreta en unidades de página de 4 KB (hasta 4 GB).
Bit D	Bit (1 bit)	Default Operand Size / Default Operation Size.	En el Descriptor de Segmento. Determina el tamaño por defecto de los operandos y direcciones. Si D=0, se asume un tamaño de 16 bits (Modo 8086). Si D=1, se asume un tamaño de 32 bits (Modo 386).
Bit P	Bit (1 bit)	Present bit (Presente).	En el Descriptor de Segmento. P=1 indica que el segmento está actualmente cargado en la memoria física. Si P=0, se genera una excepción de falta de segmento. Es fundamental para la gestión de la memoria virtual.
Bit S	Bit (1 bit)	System bit (Sistema).	En el Descriptor de Segmento. Determina el tipo de segmento. Si S=1, el descriptor define un segmento de código o datos (segmentos de aplicación/usuario). Si S=0, el descriptor define un segmento de sistema (ej. Descriptores de LDT, TSS).
Bit E	Bit (1 bit)	Executable bit (Ejecutable).	Un subtipo del bit S (S=1). E=1 indica un segmento de código (ejecutable). E=0 indica un segmento de datos.
Bit A	Bit (1 bit)	Accessed bit (Accedido).	En el Descriptor de Segmento. El procesador lo establece a A=1 automáticamente cada vez que se accede al segmento. Lo utiliza el sistema operativo para implementar políticas de reemplazo de segmentos (similares al bit de Referencia en paginación).

d. Práctica: Ejercicio de MP (Interpretación y funcionamiento) Consigna: Realice un diagrama interpretando en Modo Protegido el siguiente selector: CS=A3FF.

Interpretación del Selector CS=A3FF

Primero, convertimos el valor hexadecimal del Selector a binario de 16 bits para identificar sus campos:

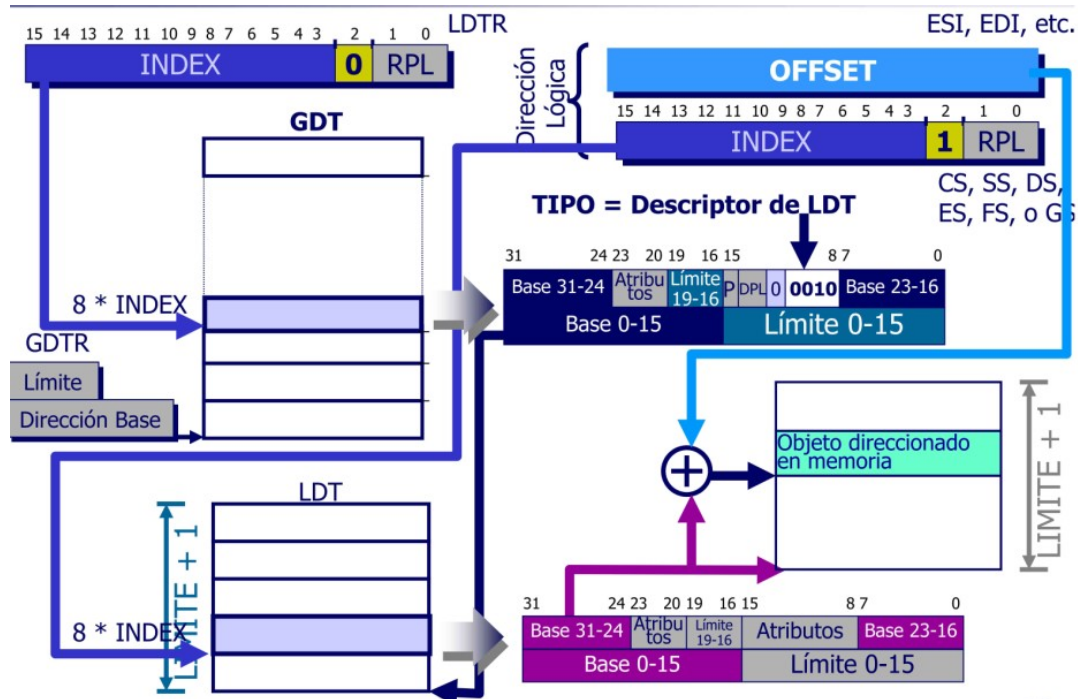
A3FF = 1010-0011-1111-1111

Campo	Bits	Valor Binario	Valor Decimal	Significado
Índice (Index)	Bits 3-15 (13 bits)	1010-0011-1111-1	A3F = 2623	Desplazamiento a la entrada 2623 de la tabla.
TI (Table Indicator)	Bit 2 (1 bit)	1	1	El descriptor está en la Tabla de Descriptores Locales (LDT).
RPL (Requested Privilege Level)	Bits 0-1 (2 bits)	11	3	Nivel de Privilegio Requerido: Ring 3 (mínimo privilegio).

Funcionamiento

1. Registro Base: Como TI=1, el procesador toma la Dirección Base de la LDT (Tabla de Descriptores Locales) del registro de sistema LDTR (el descriptor de la LDT se obtiene de la GDT, pero el acceso comienza con el LDTR).
2. Cálculo de la Dirección del Descriptor: El procesador calcula la dirección física del Descriptor de Segmento:
$$\text{Dirección Descriptor} = \text{Base de la LDT} + (\text{Índice} \times 8)$$
$$\text{Dirección Descriptor} = \text{Base de la LDT} + (2623 \times 8)$$
3. Obtención del Descriptor: Se lee el Descriptor de Segmento ubicado en esa dirección de la LDT.
4. Verificación y Carga:
 - Paso 1: Verificación de Límite (LDT): Se comprueba que el índice 2623 no exceda el límite de la LDT (almacenado en el descriptor de la LDT en la GDT).
 - Paso 2: Verificación de Presencia (P): Se verifica si el bit P=1 en el descriptor obtenido.

- Paso 3: Verificación de Privilegio (CS): El procesador realiza la verificación de privilegio para el acceso a código. La regla para cargar CS es que el RPL (3) y el CPL actual deben ser iguales al DPL (o una llamada de puerta válida).
 - Si todas las verificaciones son exitosas, el procesador carga la Dirección Base del segmento y sus Derechos de Acceso del descriptor en la parte oculta del registro CS.
5. Ejecución: El código se ejecuta en el nivel de privilegio 3, utilizando la dirección base y el límite definidos en el descriptor de la entrada 2623 de la LDT.



2) Descriptores de Segmento y Descriptores de Sistema

a. ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas.
 NOTA: No es lo mismo “tablas de descriptores” que “descriptores”.

Existen dos tipos principales de tablas de descriptores en la arquitectura 80386:

1. Tabla de Descriptores Globales (GDT - Global Descriptor Table):
 - Propósito: Contiene descriptores para segmentos que son comunes y accesibles a todas las tareas (procesos) en el sistema.
 - Contenido: Típicamente incluye los descriptores para el código y datos del Sistema Operativo (Kernel) y también los descriptores que definen las LDT de cada tarea.
 - Localización: Su ubicación y límite en la memoria son definidos por el registro GDTR (Global Descriptor Table Register).
 - Acceso: Se accede a ella cuando el bit TI (Table Indicator) del selector es TI=0.
2. Tabla de Descriptores Locales (LDT - Local Descriptor Table):
 - Propósito: Contiene descriptores para segmentos que son exclusivos de una tarea específica (proceso).

- Contenido: Contiene descriptores para el código, datos, y pila privados de esa tarea. Esto garantiza que una tarea no pueda acceder al área local de otra tarea.
- Localización: El registro LDTR (Local Descriptor Table Register) almacena un selector que apunta al descriptor de la LDT, el cual reside a su vez en la GDT.
- Acceso: Se accede a ella cuando el bit TI del selector es TI=1.

NOTA: El término "descriptores" se refiere a las estructuras de 8 bytes que definen un segmento (su base, límite y atributos) ¹⁰, mientras que "tablas de descriptores" (GDT y LDT) son las estructuras de datos en memoria (arreglos) que agrupan estos descriptores ¹¹

b. Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte el mismo. ¿Qué "tipos" de estos descriptores existen?

El bit S (System bit) es un atributo de un Descriptor de Segmento (bit 43, o bit 4 en la segunda palabra de 32 bits del descriptor 12) que define la naturaleza del segmento descrito:

- S = 1 (Descriptor de Segmento de Código o Datos):
 - Indica que el descriptor describe un segmento para aplicaciones de usuario o programas de propósito general ¹³.
 - Se utiliza para definir el código ejecutable, datos de usuario, o pilas.
 - Este tipo de descriptor puede ser uno de dos tipos primarios (definidos por el bit E): Código (E=1) o Datos (E=0).
- S = 0 (Descriptor de Segmento de Sistema):
 - Indica que el descriptor describe un segmento de control o una estructura interna utilizada exclusivamente por el sistema operativo (Kernel) ¹⁴.
 - Se utiliza para gestionar la tarea, las interrupciones y las llamadas de procedimiento de alto privilegio.

Tipos de Descriptores de Sistema ($\text{S}=0$)

Cuando el bit S=0, el campo Tipo (Type) del descriptor (bits 0-3) se utiliza para identificar estructuras de control específicas. Estos "tipos" son:

Tipo (Bits 11-8 en 16 bits)	Descripción	Uso Típico
0010	LDT (Local Descriptor Table)	Define la ubicación y el tamaño de una tabla de descriptores locales ¹⁶ .
0001 / 1001	TSS (Task State Segment)	Define un bloque de memoria que almacena todo el contexto de hardware (registros) de una tarea ¹⁷ . Es esencial para el cambio de tareas (multitarea).
Puertas (Gates)	Puertas de Llamada (Call Gates), Puertas de Interrupción, Puertas de Trampa (Trap)	Permiten el control de la transferencia de control entre segmentos de diferentes niveles de privilegio ¹⁸ .

c. Explique el concepto y funcionamiento de un “Call via Gate”.

El "Call via Gate" (Llamada a través de Puerta) es el mecanismo principal de la arquitectura 80386 que permite a un código de bajo privilegio (Ring 3) transferir control de forma controlada y segura a un código de alto privilegio (Ring 0, el Kernel), o viceversa, lo que se conoce como cambio de privilegio.

Concepto

Una Puerta de Llamada (Call Gate) es un tipo de Descriptor de Sistema ($S=0$) que actúa como un punto de entrada seguro al código del sistema operativo. La puerta no contiene el código en sí, sino la información necesaria para que el procesador realice la transferencia de control de forma controlada.

Funcionamiento (El Cambio de Privilegio)

1. La Solicitud: Un programa de aplicación de usuario (CPL = 3) ejecuta una instrucción de llamada lejana (CALL FAR) utilizando un Selector que apunta a una Puerta de Llamada en la GDT o LDT.
2. Verificación de Privilegio: El procesador realiza tres verificaciones de seguridad:
 - RPL vs DPL (Puerta): El Nivel de Privilegio Requerido (RPL) del selector de la puerta debe ser menor o igual que el DPL de la Puerta¹⁹. (Es decir, el llamante debe tener suficiente privilegio para usar la puerta).
 - CPL vs DPL (Puerta): El Nivel de Privilegio Actual (CPL) del llamante debe ser menor o igual que el DPL de la Puerta.
 - CPL vs DPL (Segmento de Destino): El DPL del segmento de código de destino (definido dentro de la Puerta) debe ser numéricamente menor (más privilegiado) que el CPL.
3. Transición (Si se cumplen las condiciones):

- Cambio de Pila: Si el nivel de privilegio es diferente (ej. de Ring 3 a Ring 0), el procesador cambia automáticamente a una pila más privilegiada (la pila del Kernel) definida en el TSS de la tarea. La información de la pila anterior del usuario y los parámetros de la llamada se copian a la nueva pila.
- Transferencia de Control: El procesador carga el Selector del Segmento de Destino (que está almacenado dentro de la Puerta) en el registro CS y salta a la Dirección Base del segmento de destino (el código del Kernel).

Este mecanismo garantiza que el código de alto privilegio (Kernel) solo pueda ser accedido en puntos de entrada predefinidos y con estrictas verificaciones de privilegio, protegiendo así la integridad del sistema.

3) Segmento de Estado de la Tarea (TSS)

a. ¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?

El objetivo principal del TSS, relacionado con el uso del procesador y la conmutación de tareas, es garantizar una transición rápida, eficiente y segura entre las tareas o procesos que se ejecutan en el sistema.

Lo que se busca es:

- Salvar el Contexto del Procesador: Cuando el sistema operativo (SO) decide cambiar de la Tarea A a la Tarea B, el TSS de la Tarea A almacena todo el contexto de hardware de la Tarea A. Esto incluye todos los registros de propósito general (EAX, EBX, etc.), registros de segmento (CS, DS, etc.), el registro de flags (EFLAGS) y el puntero de instrucción (EIP).
- Aislamiento y Reproducibilidad: Al guardar el contexto completo en el TSS, se asegura que cuando la Tarea A sea reanudada posteriormente, lo haga exactamente desde el estado en que fue interrumpida, con sus recursos y datos intactos. Cada tarea tiene su propia área de memoria exclusiva para su código y datos.
- Cambio Atómico de Tarea: El procesador 80386 puede realizar la conmutación de tareas (cargar el nuevo estado desde el TSS de la Tarea B) en una sola instrucción atómica (por ejemplo, con un salto o llamada lejana a un descriptor de TSS). Esto es crucial para la alta velocidad y seguridad de la multitarea.

En resumen, el TSS permite que el procesador "recuerde" dónde se quedó cada tarea para poder alternar rápidamente entre ellas, dando la ilusión de ejecución simultánea.

b. Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.

Funcionamiento del TSS

El TSS se utiliza de dos formas principales en la conmutación de tareas:

1. Activación de Tarea (Carga): Cuando se activa una nueva tarea (Tarea B), el procesador lee el contenido del TSS de la Tarea B para restaurar los registros internos.

2. Suspensión de Tarea (Almacenamiento): Cuando la tarea actual (Tarea A) se suspende, el procesador guarda el estado actual de sus registros en el TSS de la Tarea A antes de cargar el estado de la Tarea B.

Proceso de Conmutación de Tareas (Implicito):

1. El SO ejecuta una instrucción que apunta al Descriptor de TSS de la nueva tarea (por ejemplo, JMP o CALL a un selector de TSS).
2. El procesador: a. Guarda el contexto actual (registros, CS, EIP, etc.) en el TSS de la tarea saliente. b. Lee el contexto de la nueva tarea desde su TSS (registros, CS, EIP, etc.). c. Actualiza el registro TR (Task Register) para que apunte al selector del Descriptor de TSS de la nueva tarea. d. La nueva tarea se reanuda desde el punto donde fue suspendida.

Esquema y Diagrama de Interpretación del TSS

El TSS es una estructura de datos de al menos 104 bytes (para la versión de 32 bits, 80386). Contiene campos clave para la gestión de tareas, especialmente para el cambio de privilegio (Stack Pointers Ring 0, 1, 2) y la paginación de E/S (I/O Map Base Address).

Campo	Bits	Descripción	Propósito
Link (Previous Task Link)	16	Selector	Apunta al TSS de la tarea que llamó a la tarea actual (usado para IRET).
ESP0, SS0	32, 16	Puntero a Pila de Ring 0	Define la Pila que se utilizará cuando esta tarea cambie de CPL > 0 a Ring 0 (Kernel).
ESP1, SS1	32, 16	Puntero a Pila de Ring 1	Define la Pila de Ring 1 para el cambio de privilegio.
ESP2, SS2	32, 16	Puntero a Pila de Ring 2	Define la Pila de Ring 2 para el cambio de privilegio.
CR3	32	Registro de Control 3	Contiene la dirección base del Directorio de Páginas (usado para la memoria virtual)4444.
EIP, EFLAGS	32	Registros de Control	Almacena el puntero de instrucción y el registro de flags para reanudar la tarea.
Registros Generales	32	EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP	Almacena el estado de los registros de propósito general.
Registros de Segmento	16	ES, CS, SS, DS, FS, GS	Almacena el estado de los registros de segmento.
I/O Map Base Address	16	Dirección Base del Mapa de E/S	Controla el acceso a puertos de E/S por parte de esta tarea.

El TSS es esencialmente la "ficha de identidad" y el "estado de memoria" de una tarea dentro de un entorno multitarea.

4) Paginado

a. Explique conceptualmente para qué sirve el proceso de paginado y como se relaciona con el sistema de segmentado.

Concepto del Paginado

El paginado sirve para conseguir direccionar una gran cantidad de memoria , permitiendo que el programador disponga de un espacio de direccionamiento mucho mayor que el disponible físicamente.

Conceptualizando:

- **Memoria Virtual:** El paginado permite que cada programa perciba que tiene un espacio de memoria lineal grande y contiguo (el Espacio de Dirección Lineal), incluso si el espacio físico (RAM) disponible es menor y está fragmentado. El sistema operativo utiliza el disco como extensión de la RAM para almacenar datos y código ausentes.
- **Gestión de Memoria No Contigua:** Divide el espacio de dirección lineal en bloques de tamaño fijo llamados páginas (típicamente 4 KB), y la memoria física (RAM) en bloques del mismo tamaño llamados marcos de página (Page Frames). Esto permite almacenar páginas no contiguas en el espacio físico.
- **Aislamiento:** Cada tarea puede tener su propio conjunto de tablas de paginación, lo que asegura que no pueda acceder a las páginas de otra tarea (aislamiento de memoria).

Relación con el Sistema de Segmentado

El paginado opera inmediatamente después de la segmentación. El flujo completo de traducción de direcciones en el Modo Protegido es:

1. **Segmentación (Primera Capa):** El procesador convierte la Dirección Lógica (Selector:Offset) en una Dirección Lineal de 32 bits.
 - **Propósito:** Proporcionar estructuras lógicas de código/datos (segmentos) y aplicar la protección de acceso (privilegio).
2. **Paginado (Segunda Capa):** La Dirección Lineal de 32 bits generada por la segmentación se convierte en la Dirección Física de la memoria RAM.
 - **Propósito:** Implementar la Memoria Virtual y la gestión de la memoria física no contigua.

Si el bit PG (Paginado) en el registro de control CR0 está desactivado (PG=0), la Dirección Lineal se convierte directamente en la Dirección Física. Si PG=1, se activa el mecanismo de paginado.

b. ¿Qué hace la unidad de paginado? ¿Cómo lo hace ?

La Unidad de Paginado (o Unidad de Gestión de Memoria - MMU) toma la Dirección Lineal producida por la segmentación y la traduce a una Dirección Física.

¿Qué hace la Unidad de Paginado?

La Unidad de Paginado es responsable de:

1. Traducción de Dirección: Convertir la Dirección Lineal de 32 bits en una Dirección Física de la RAM.
2. Manejo de Fallos de Página: Si la página requerida no se encuentra en la RAM (bit P=0), la unidad genera un evento llamado Fallo de Página (Page Fault), que el Sistema Operativo debe manejar.
3. Protección a Nivel de Página: Aplica permisos (lectura/escritura) y niveles de privilegio a nivel de página, sumándose a la protección por segmento.

¿Cómo lo hace? (Mecanismo de Tablas de Dos Niveles)

La traducción de la Dirección Lineal se realiza a través de un esquema jerárquico de dos niveles de tablas:

1. Descomposición de la Dirección Lineal: La Dirección Lineal de 32 bits se divide en tres campos:
 - Directorio de Páginas (Page Directory Index): Bits 22-31 (10 bits).
 - Tabla de Páginas (Page Table Index): Bits 12-21 (10 bits).
 - Desplazamiento (Offset): Bits 0-11 (12 bits).
2. Nivel 1: Directorio de Páginas (Page Directory):
 - El registro de control CR3 contiene la Dirección Base del Directorio de Páginas 8888 (PD).
 - El procesador usa el Directorio de Páginas (10 bits) para seleccionar una de las 1024 entradas del PD.
 - La entrada del PD apunta a la Dirección Base de una Tabla de Páginas.
3. Nivel 2: Tabla de Páginas (Page Table):
 - Usando la Dirección Base de la Tabla de Páginas, el procesador usa el campo Tabla de Páginas (10 bits) para seleccionar una de las 1024 entradas de la Tabla de Páginas (PT).
 - La entrada de la PT contiene la Dirección Base del Marco de Página (Page Frame) de 20 bits.
4. Generación de la Dirección Física:
 - La Dirección Base del Marco de Página (20 bits) se concatena con el Desplazamiento (12 bits) para formar la Dirección Física de 32 bits.

Para optimizar este proceso (evitar dos accesos a memoria por cada acceso), el procesador utiliza el Translation Lookaside Buffer (TLB), una caché interna que almacena las traducciones de páginas usadas recientemente.